aleatorio. Asimismo, se separaron las palabras en subunidades mediante un algoritmo BPE para finalmente ser traducidas a cuatro espacios vectoriales.

Dichas operaciones iniciales de preprocesamiento fueron necesarias para evitar que los datos contengan elementos que puedan entorpecer el proyecto. Por ejemplo, si se hubiesen dejado a kapa y Kapa (ardilla) como dos elementos distintos, el tokenizador los habría codificado de forma distinta. Esto a su vez llevaría dos representaciones distintas en el espacio vectorial que podrían perjudicar el aprendizaje del modelo. Por otro lado, la elección de un tokenizador a nivel de subpalabra permitió trabajar de una mejor forma con la naturaleza aglutinante del Iskonawa. Se sugiere como trabajos futuros explorar el impacto de un tokenizador morfológico para la separación en subpalabras.

La aumentación a su vez impacta favorablemente en el futuro del desarrollo del proyecto, pues aproximadamente se duplicó la cantidad de registros con los que se contaba. Este incremento fue realizado sin alterar excesivamente el significado y estructura de los registros como se describió en la sección. El uso de la gramática Iskonawa (Zariquiey, 2015) fue vital para decidir qué morfemas mutar y bajo qué condiciones hacerlo. En ese sentido, se recomiendo que futuras tareas de aumentación en otras lenguas de bajos recursos hagan uso de este conocimiento lingüístico; ya sea mediante el uso de una gramática en dicha lengua o el uso de anotaciones morfológicas. Adicionalmente, se evidenció que contar con anotaciones lingüísticas respecto a las dependencias en los registros hubiera permitido utilizar otras técnicas de aumentación, como las relacionadas a los árboles de dependencia.

Finalmente, la tarea de clusterización permitió tener una idea de qué métodos de vectorización fueron los más adecuados. Si bien las métricas indicaron que los embeddings conseguidos a partir del método de embeddings bilingües anclados fueron los que permitieron un mejor agrupamiento, la utilidad real de este espacio vectorial será verificada en las siguientes etapas del proyecto.

Capítulo 6. Definición y desarrollo de los modelos generativos

Como se comentó en el Capítulo 1, la investigación en NLP en lenguas de bajos recursos suele ser escasa en comparación a la realizada en las lenguas más habladas. Esta limitada cantidad

de estudios se hace más notoria cuando se recorta el campo a la subárea de Natural Language Generation (NLG); área en la que se enfoca el objetivo de esta tesis. En este capítulo se explorarán algunas de estas técnicas y arquitecturas encontradas en la literatura para hacer frente a la escasez de datos en el entrenamiento de modelos de NLG. Se explicará brevemente en qué consiste cada una para luego probarlas y entrenarlas con el corpus en Iskonawa. Finalmente, se compararán entre sí de acuerdo a las métricas elegidas.

6.1 Natural Language Generation en lenguas de bajos recursos

Las técnicas encontradas tienen como características en común ciertos aspectos: una arquitectura basada en Transformers, uso de modelos pre-entrenados y aplicación de transferencia de aprendizaje (revisar sección 1.3). Es decir, buscan utilizar el conocimiento adquirido en el pre-entrenamiento con lenguas de altos recursos para lograr un mejor desempeño en tareas con lenguas de bajos recursos. A continuación, se ofrece un breve resumen de las técnicas consideradas para este estudio:

- **ZmBART:** "Unsupervised Cross-lingual Transfer Framework for Language Generation" (Maurya et al., 2021)

  En este artículo se propone ZmBART, un framework para la transferencia de conocimiento que no necesita de corpus paralelos. En el artículo, los autores se enfocan en resolver cuatro tareas de NLG: resumido de textos, generación de preguntas, generación de titulares y generación de distractores.

  El funcionamiento planteado es el siguiente: Se empieza con un modelo pre-entrenado multilingüe mBART. Este es un modelo secuencia a secuencia basado en Trasformers originalmente propuesto en (Y. Liu et al., 2020). Luego, se pre-entrena aún más este modelo con una tarea auxiliar para adaptarlo a la generación de lenguaje y a la lengua objetivo. A continuación, se ajusta el modelo con datos en inglés de una tarea en específico (por ejemplo, generación de titulares). Finalmente, se prueba el modelo en la lengua objetivo y la tarea especificada para lograr la generación de lenguaje. Esta secuencia puede ser observada visualmente en la Figura 10.

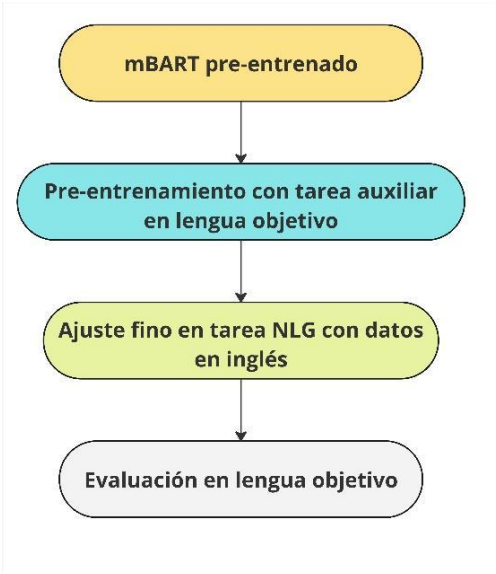

Figura 10. Funcionamiento del framework ZmBART. Adaptado de *(Maurya et al., 2021)*

- **Meta-X_{NLG}:** "Meta-Learning Approach Based on Language Clustering for Zero-Shot Cross-Lingual Transfer and Generation" (Maurya y Desarkar, 2022)
  Este enfoque combina meta-aprendizaje (aprender a aprender) y clusterización de lenguas para conseguir una mejor transferencia de aprendizaje a nuevas lenguas. El algoritmo de meta-aprendizaje usado es MAML (Model Agnostic Meta-Learning), propuesto en (Finn et al., 2017). Este algoritmo busca aprender la estructura en común de una serie de tareas con el fin de que, en nuevas tareas con una estructura similar, el modelo pueda aprender rápidamente y sin necesidad de tantos datos.

  El flujo de trabajo propuesto es el siguiente: Primero, se agrupan las lenguas a utilizar siguiendo el enfoque de (Oncevay et al., 2020) que combina representaciones tipológicas y aprendidas en tareas. En base a estos grupos, se encuentran las "lenguas centroides" de cada cluster. El estudio manifiesta que 3 clusters fue el número que obtuvo mejores resultados. Luego, se toma un modelo multilingüe pre-entrenado; los autores consideraron al modelo mT5. Además, se pre-entrena este modelo de manera no supervisada en una muestra de las lenguas que se usarán. Luego, se ajusta el modelo con datos de una lengua en altos recursos para una tarea en específico. A continuación, la etapa de meta-aprendizaje comienza, donde se usan las "lenguas centroide" identificadas anteriormente con sus respectivos conjuntos de datos. Finalmente, el modelo es evaluado en lenguas no centroides en la generación de lenguaje. Se puede

añadir una última capa de ajuste fino si es que se tienen datos en la lengua objetivo. Este flujo puede ser apreciado gráficamente en la Figura 11.

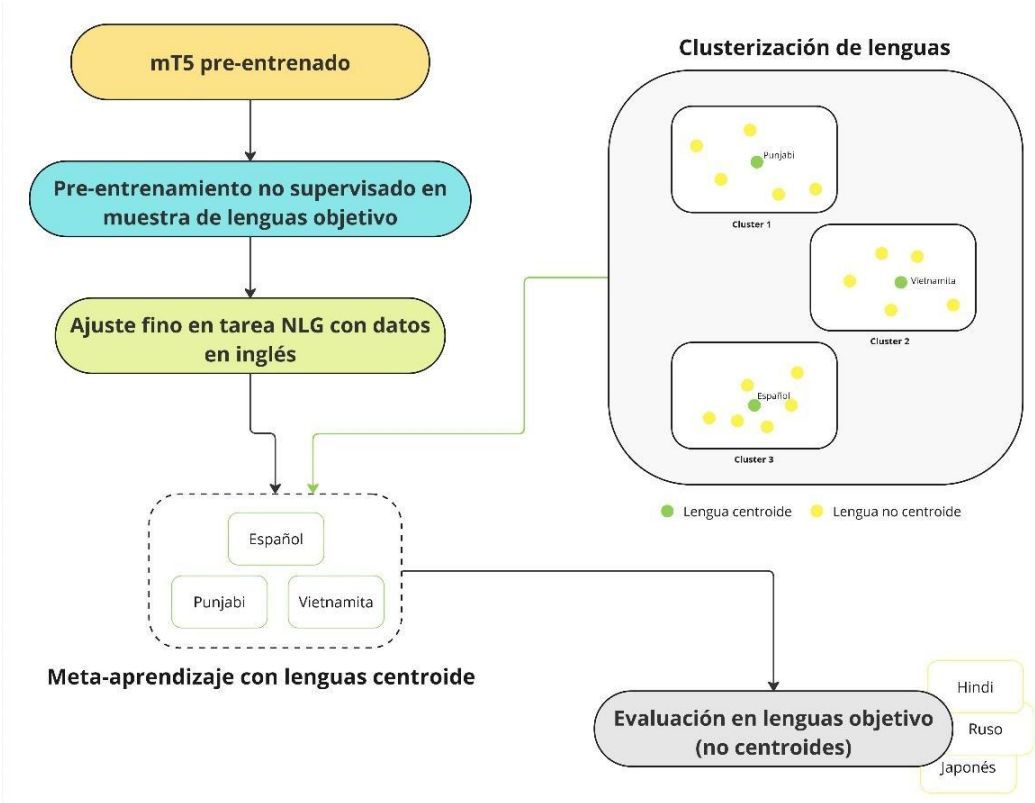

Figura 11. Funcionamiento del framework Meta-X_{NLG}. Adaptado de *(Maurya y Desarkar, 2022)*

- **XNLG: "**Cross-Lingual Natural Language Generation via Pre-Training" (Chi et al., 2019)

  Este estudio entrena un modelo secuencia a secuencia mediante distintas tareas de pre-entrenamiento para permitir la generación de lenguaje en varias lenguas. Como en las otras técnicas, se utiliza una arquitectura de Transformers como la base. Los autores toman al modelo pre-entrenado multilingüe XLM entrenado en 15 idiomas (Lample y Conneau, 2019) para inicializar los pesos del codificador y el decodificador del modelo a pre-entrenar.

  El proceso planteado empieza con la inicialización de los pesos del modelo en base al modelo XLM mencionado antes. Luego, se entrena al codificador con dos tareas: modelado de lenguaje enmascarado (MLM) y modelado de lenguaje enmascarado multilingüe (XMLM). Por su parte, el decodificador también se entrena con otras dos tareas ("Denoising Auto-Encoding" y "Cross-Lingual Auto-Encoding") para asegurar

que pueda generar lenguaje cuando luego se a ajustado con otras tareas. Este modelo es finalmente ajustado con data de tareas de generación de preguntas y generación de resúmenes (abstractive summarization).

En adición a estas técnicas enfocadas en la transferencia de aprendizaje, se probará el entrenamiento desde cero de un modelo LSTM y un modelo T5. Los detalles de estos procedimientos se detallan en la siguiente sección.

6.2 Resultados alcanzados

6.2.1 Uso y adaptación de las técnicas descritas

Los resultados descritos en esta sección responden al resultado esperado R3.1. En base a los métodos consultados y lo sugerido por el experto en el Anexo C, se procedió con el entrenamiento. Todos estos modelos fueron entrenados mediante potencia de cómputo en la nube. En cada uno de los experimentos que se describen a continuación se detalla el equipo usado, así como los parámetros de entrenamiento. En general, no se agregó en estos modelos la última capa de ajuste fino para una tarea en específico debido a que no es el objetivo del estudio comparar su performance en dichos dominios.

6.2.1.1 ZmBART

Antes de empezar el entrenamiento, se configuró un ambiente de Conda con la lista de librerías y dependencias indicada en el repositorio del artículo[8]. Siguiendo lo indicado en el artículo original, se empezó descargando los pesos del modelo pre-entrenado mBART (CC25). Luego, los datos en Iskonawa fueron convertidos para cumplir con la tarea auxiliar propuesta en el artículo. Esta consiste en, dado un conjunto de oraciones, generar un pequeño porcentaje (20%) de esas oraciones. Para lograr esto, se utilizó el script de conversión también provisto en el repositorio.

A continuación, estos datos fueron binarizados utilizando una versión modificada de la librería Fairseq también incluida en el repositorio. Se separó un 80% de los datos como el conjunto de entrenamiento, los cuales fueron alimentados al modelo para sen entrenado con las siguientes características: La arquitectura fue mBART-large. Se empleó como función de pérdida el criterio de entropía cruzada suavizada con un factor de suavizado de etiquetas de 0.2. El optimizador utilizado fue Adam, con un epsilon de 1e-06. La tasa de aprendizaje siguió un

[8] https://github.com/kaushal0494/ZmBART

esquema de decaimiento polinomial; se comenzó con un valor de 3e-05, y un calentamiento de 2500 actualizaciones. Se tuvo también una tasa de dropout del 30%. Se utilizaron 2048 tokens por batch. Los valores previamente descritos fueron sugeridos en el artículo original y se conservaron para este experimento. El entrenamiento se realizó por 150 épocas y se fue guardando cada 10 épocas un checkpoint para no perder el progreso. El acelerador utilizado fue una GPU A4000 y se utilizó PyTorch.

6.2.1.2 Meta-X_{NLG}

De manera similar al artículo anterior, se configuró un ambiente en Conda con la lista de librerías y dependencias indicada en el repositorio[9] del artículo. Esta vez se empezó con el modelo pre-entrenado mT5 que obtuvieron los autores luego de pre-entrenarlo con otras 30 lenguas. Como se indica en el artículo, el paso inicial fue pre-entrenar este modelo en la lengua objetivo. Para realizar este primer paso, la tarea sugerida por el autor era la misma descrita en el método ZmBART, pero esta vez con una arquitectura mT5. De manera similar a la sección 6.1.1, se consideró el 80% de los datos para el conjunto de entrenamiento y se siguieron las mismas consideraciones de preprocesamiento.

Los parámetros en el entrenamiento fueron los siguientesse utilizó un factor de suavizado de etiquetas de 0.2. El optimizador utilizado fue Adam, con un epsilon de 1e-06. Al igual que en ZmBART, la tasa de aprendizaje siguió un esquema de decaimiento polinomial. Se comenzó con un valor de 3e-05 y un calentamiento de 2500 actualizaciones. Se aplicó una tasa de dropout del 30%. Para el tamaño del batch, se definieron 64 ejemplos por dispositivo debido a las restricciones de la memoria. El proceso se ejecutó durante 150 épocas bajo la librería PyTorch y un acelerador RTX 3090. Se guardaron puntos de control cada 5000 pasos hasta llegar a las 150 épocas de entrenamiento.

A continuación, dado que el modelo preentrenado ya había sido desarrollado por los autores, la etapa de meta-aprendizaje no fue necesaria. En otras palabras, el enfoque consistía en mejorar el modelo a través del meta-aprendizaje utilizando lenguas centroide, para luego evaluarlo en lenguas no centroide (en nuestro caso, el Iskonawa). Como este proceso ya se había completado previamente, únicamente fue necesario probar la generación en Iskonawa.

[9] https://github.com/kaushal0494/Meta_XNLG

Para identificar mejor el impacto de las otras lenguas, se intentó replicar el método de clusterización mencionado en el artículo, publicado originalmente en (Oncevay et al., 2020), para visualizar dónde se ubicaba el Iskonawa. Sin embargo, dicho trabajo fue realizado con corpus paralelos de las 53 lenguas que clusterizaba. Es decir, si queríamos introducir al Iskonawa en esa agrupación, haría falta conseguir un corpus paralelo al ya existente, pero en Iskonawa o, conseguir corpus paralelos en todas las otras lenguas al que se tiene en Iskonawa. Dadas las limitaciones de tiempo y de potencia de cómputo de este proyecto de tesis se decidió no seguir con esta reproducción.

6.2.1.3 XNLG

En este método se entrena de forma separada al codificador y al decodificador. Se planificó seguir lo indicado en el repositorio[10] del artículo y se prepararon los datos en Iskonawa para el formato que se requería. Al comenzar el entrenamiento, se encontraron problemas de compatibilidad debido a que se esperaba el uso de la versión 1.1 de PyTorch, pero se contaba con una versión más actual. Ante ello, se aplicaron las técnicas de mitigación descritas en el Anexo B: Plan de Proyecto y se utilizaron versiones más antiguas de las librerías para asegurar la compatibilidad.

No obstante, la versión de PyTorch requerida (1.1) soportaba como máximo a la versión 10 del Cuda Toolkit (un conjunto de utilitarios para poder utilizar un GPU como acelerador del entrenamiento). Las GPUs con las que se estaba entrenando necesitaban como mínimo la versión 11 del Cuda Toolkit. Se intentó buscar aceleradores más antiguos, pero no se encontraron versiones compatibles al momento de la redacción de este capítulo en los proveedores cloud con los que se estaba trabajando. Por ello, se decidió continuar sin probar esta técnica para no retrasar al proyecto.

6.2.1.4 LSTM

Como se describió antes, este modelo fue planteado como una línea base sobre la cual comparar el resultado de otros modelos. La arquitectura definida para este modelo fue la siguiente:

- Capa de embeddings. Para convertir los tokens de entrada en vectores. Esta fue inicializada con los embeddings obtenidos en el capítulo anterior
- Capa LSTM. Dos capas ocultas de tamaño 128, Dropout de 0.3 y modo bidireccional
- Capa de normalización

[10] https://github.com/CZWin32768/XNLG

- Capa Fully Connected. Para generar la distribución de probabilidad. El tamaño es el tamaño del vocabulario de los embeddings
- Capa Dropout con un factor de 30%

Los datos en Iskonawa fueron preprocesados siguiendo lo descrito en el capítulo anterior sin llegar a la etapa de vectorización, pues dicho proceso ya estaba incluido en la primera capa del modelo. Se utilizaron las clases Dataset y DataLoader de PyTorch para facilitar el aprovisionamiento de los datos al modelo. También se utilizó el 80% de los datos como conjunto de entrenamiento

Los embeddings preentrenados cargados en la capa de embeddings se congelan para evitar que se actualicen durante el entrenamiento. El entrenamiento se ejecutó con el optimizador con Adam bajo una tasa de aprendizaje de 0.001. La función de pérdida utilizada fue la entropía cruzada y se consideró un tamaño de batch de 256. Se entrenó durante 150 épocas y, como en casos anteriores, se fue guardando un checkpoint cada 10 épocas para evitar la pérdida del modelo. Este modelo fue entrenado utilizado una TPU 2-8 de Google Cloud. Para ello, se utilizó el framework y librería PyTorch/XLA.

6.2.1.5 T5

Se entrenó también un modelo T5 ajustado para la generación de texto en Iskonawa. Los pasos seguidos se basaron en una parte de la investigación de (Ulčar y Robnik-Šikonja, 2023). Dicha investigación tiene un repositorio[11] público del cual se utilizaron fragmentos de código para el entrenamiento. El porcentaje de datos utilizados fue también del 80% para el entrenamiento. Para el entrenamiento se consideraron las tareas provistas en el repositorio, las cuales consistían de span corruption y i.i.d. denoising (ambas consistentes en limpiar el ruido de un token seleccionado aleatoriamente). Estas se gestionaron a través del registro de tareas de SeqIO, donde se definió el vocabulario generado por el tokenizador SentencePiece previamente entrenado. La arquitectura elegida fue un t5_small con las siguientes características:

- El tamaño del espacio de representación de las capas internas es de 512 dimensiones
- El modelo está compuesto por 8 capas en el codificador y el decodificador.
- El tamaño de la capa feed-forward es de 1024 dimensiones.
- El número de cabezas en el mecanismo de atención es de 6.

[11] https://github.com/MatejUlcar/SloT5-tools

El entrenamiento también se realizó guardando checkpoints y con un acelerador GPU RTX 3090 en la nube. Se utilizó TensorFlow y no PyTorch debido a que el código utilizado se basaba en el módulo T5 Mesh Transformer, el cual tenía como dependencia a TensorFlow

6.2.2 Cálculo de métricas

Luego de entrenar los modelos indicados, se llevaron los pesos a una máquina virtual con un acelerador A4000. Se prepararon los datos y se elaboraron funciones para evaluar el rendimiento de los modelos con tres métricas intrínsecas automáticas y cumplir así con el resultado esperado R3.2.

- **Perplejidad**. Originalmente propuesta en (Jelinek et al., 1977), en esencia mide qué tan bien un modelo probabilístico predice una muestra. En el contexto del procesamiento de lenguaje natural, la perplejidad puede entenderse como el nivel de confianza que tiene el modelo para generar la siguiente palabra en una secuencia. Retorna un valor positivo y un valor bajo de perplejidad bajo indicará que el modelo tiene más certeza en sus predicciones de las secuencias (*Perplexity of fixed-length models*, s. f.).
  Es importante destacar que esta métrica no intenta evaluar el contenido generado y no provee información respecto a si este es de calidad o no. Solo se enfoca en la certeza de generación de la siguiente secuencia.
- **Distinct-N**. Esta métrica, presentada en (J. Li et al., 2016) mide la diversidad del contenido generado a nivel de oraciones. Se realiza una simple división entre la cantidad de n-gramas únicos en una oración y el total de n-gramas en la oración para evitar que oraciones largas tengan una mejor puntuación. Al igual que la anterior, esta métrica tampoco intenta medir la calidad del contenido generado respecto a una muestra real; solo se enfoca en la diversidad. Para este estudio, se consideraron bigramas y trigramas a nivel de caracteres. Un valor más alto representa una mayor diversidad.
- **MAUVE**. Propuesta por (Pillutla et al., 2021), esta métrica evalúa la calidad de las oraciones generadas al comparar la distribución de estas salidas con la distribución de una muestra de datos reales. Es decir, MAUVE sí intenta dar una idea de qué tan parecido es el contenido generado al contenido real. Consigue esto al calcular la divergencia de Kullback-Leibler de las distribuciones llevadas a un espacio vectorial más pequeño. Sus posibles valores están entre 0 y 1. Un valor MAUVE alto indica que el texto generado es más cercano a las distribuciones humanas. Se menciona también que esta métrica es más útil en un sentido comparativo. Es decir, por sí misma no ofrece

mucha información; pero al calcularla en dos o más modelos distintos sí permite identificar cuál genera mejores resultados.

Cabe destacar que, por defecto, MAUVE utiliza el espacio vectorial del modelo pre-entrenado GPT-2. Como trabajo futuro sería relevante identificar cómo afecta a esta métrica la elección de un espacio vectorial más cercano al Iskonawa. Para este estudio se tomó una muestra de 300 oraciones aleatorias de los datos reales en Iskonawa.

En todos los casos, se calcularon las métricas 5 veces y se reporta en la Tabla 16 el promedio de estas cifras. Se resaltan en negrita los mejores valores obtenidos. En la Tabla 17 se muestran algunos ejemplos del contenido generado por los modelos. Cabe destacar que, si bien estas métricas dan una idea del performance de los modelos, no nos permiten realizar una conclusión respecto a la calidad del contenido.

Tabla 17
Métricas automáticas de evaluación de los modelos. No hay un claro modelo sobresaliente

Modelo	**Perplejidad**	**Dinstinct-2**	**Dinstinct-3**	**MAUVE**
ZmBART	**43,4085**	0,0382	0,1181	0,0316
MetaXNLG	53,8176	0,0657	0,2042	**0,5293**
LSTM	2473,0986	0,0713	**0,2672**	0,4121
T5	169,5509	**0,0723**	0,1916	0,0090

Tabla 18
Ejemplos de generación de texto. A simple vista, el modelo T5 sufre de más repetición y poca variedad

Modelo	Ejemplo
ZmBART	bia oa ma han bain on we ch chotakashamai hono ha we han askati shoko noi bia oa ma han te bain on onai
MetaXNLG	poa toai pani iki riki pani ca ri tsanwai ha pani poa rabé noi ri we

LSTM	chin eahkair ayhepanhe hmhe pio maihonkeirtimahe abohe birahe yoike rain keyoairrokeirhe behe rakahe
T5	haweri oha oha bobai meskobo oha neska bobai onahakua bires oha oha bobai

6.3 Discusión

En este capítulo se presentaron las arquitecturas de modelo elegidas y las técnicas de entrenamiento probadas. Estas fueron elegidas en base a su enfoque para trabajar con lenguas de bajos recursos. Los resultados obtenidos muestran que todos los modelos logran producir contenido. Además, se eligió calcular métricas intrínsecas que den una idea de qué tan diverso es el contenido generado y qué tanto podría acercarse a uno producido por humanos. En ese sentido, las métricas elegidas indican que el contenido producido no es muy diverso. Sin embargo, como se mencionó en la sección anterior, no se puede llegar a una conclusión respecto a la utilidad del contenido generado debido al desconocimiento del autor sobre la lengua y la inexistencia de algún método de validación o traducción automática. Este problema será atendido en el siguiente capítulo, donde se diseñará el método de validación para facilitarle el trabajo al experto en la lengua y así identificar realmente si el texto es útil.

Al comparar los resultados obtenidos con investigaciones previas, se puede observar que las métricas elegidas dan peores resultados en comparación a otros en la perplejidad (por ejemplo, GPT-2 tiene una perplejidad de 19.43). Resulta imposible comparar con estudios anteriores, pues no existen intentos de generación de lenguaje en la lengua Iskonawa ni en lenguas cercanas. Lo identificado hasta ahora parece indicar que los resultados conseguidos no son favorables, dado que ningún modelo resalta en más de una métrica. El modelo LSTM simple elegido como línea base no es superado absolutamente por ninguna de las alternativas utilizadas. En ese sentido, se considera que un enfoque más adecuado hubiera sido priorizar tareas de entendimiento del lenguaje natural (Natural Language Understanding – NLU). Estas tareas son más viables de evaluar con las anotaciones lingüísticas de los datos que se tienen y representan una base también en el desarrollo de NLP en lenguas en peligro en el Perú. Adicionalmente, al momento de la redacción de este capítulo se identificó un artículo publicado

en Octubre de 2024 (Yong et al., 2024) que busca generar texto en lenguas con bajos recursos mediante el uso de lexicones bilingües. Esta opción será explorada más a fondo en las conclusiones, donde se tendrán ya los resultados del Objetivo específico 4 y se evaluará el mejor curso de acción a seguir.

Adicionalmente, existen ciertas limitaciones que deben ser consideradas. En primer lugar, la optimización de hiperparámetros fue limitada por las restricciones presupuestarias y temporales del proyecto. Dado que solo se contaba con una pequeña cantidad de créditos para realizar el entrenamiento en la nube y el tiempo asignado al proceso de entrenamiento fue de unas 3 semanas, no se realizó una extensiva búsqueda de los mejores parámetros de entrenamiento y se utilizaron los sugeridos en cada uno de los artículos de referencia. Además, aspectos como el impacto del uso de otros espacios vectoriales en la métrica MAUVE no fueron abordados en la presente investigación, lo que podría haber influido en los resultados obtenidos. Finalmente, a modo de lección aprendida durante el desarrollo de este capítulo, se resalta el trabajo adicional que conlleva utilizar un acelerador tipo TPU. Se utilizó este tipo de acelerador en combinación con GPUs debido a que se consiguió un mes gratuito de uso de TPU en la plataforma Google Cloud y se intentó aprovechar para reducir parte de los costos. Sin embargo, dado el trabajo adicional que representó adaptar el código de entrenamiento para utilizar este hardware, las dependencias y conflictos entre versiones debido a la necesidad de librerías especializadas y la propia inexperiencia del autor, se incurrió en un mayor tiempo que pudo haber ralentizado el desarrollo del proyecto. En ese sentido, se recomienda considerar estos factores al momento de elaborar el cronograma de actividades si es que se planea utilizar TPU.